

SecurityNode

Application Note

Controller / Application Node Design

Final Project of Electronic Design

June 17, 2026

Abstract

This application note documents the **design** of the SecurityNode controller/application node. The node is implemented on the ESP32-C3-WROOM-02-N4 and is responsible for sensor monitoring, alarm control, vision-module coordination, and host communication. Because the project is still at the design stage and no assembled hardware is available, this note focuses on architectural choices, electrical interfaces, requirements and traceability, a formal state machine, the UART protocol, and a theoretical verification plan. Measured bench data such as oscilloscope waveforms, thermal tests, and power-consumption plots are intentionally excluded.

Contents

1	Purpose and scope	3
2	Design assumptions	3
3	Design artifacts	4
4	Requirements	5
5	Traceability matrix	5
6	External interfaces	6
6.1	Pinout	6
6.2	UART interface parameters	7
7	Electrical design summary	8
7.1	Power architecture	8
7.2	Analytical power budget	8
7.3	Output drive	8
7.4	Schematic preview	8
7.5	PCB design review notes	8
8	State machine	9
8.1	State definitions	9
8.2	State-transition diagram	9
8.3	State behavior table	10

9	UART protocol interface	10
9.1	Frame format	11
9.2	Command set	11
9.3	Response handling	11
9.4	Timeout and retry rules	11
10	Firmware design sketch	12
11	Theoretical verification plan	12
11.1	Expected behavior and design targets	12
11.2	Analytical verification	12
11.3	Schematic and PCB verification	13
11.4	Excluded verification activities	13
12	Summary	13

1 Purpose and scope

This application note has three objectives:

1. Capture the design intent of the SecurityNode application/controller node.
2. Provide enough detail for a peer review of the firmware architecture, electrical interfaces, and communication protocol before hardware is built.
3. Define the verification activities that can be performed analytically or by inspection, given that no physical prototype exists yet.

The note covers:

- Design assumptions and constraints.
- Design artifacts produced by the project.
- Functional requirements and a traceability matrix.
- External interfaces and a formal pinout.
- Electrical design summary and review notes.
- Finite-state machine with a camera-availability flag.
- UART protocol interface between the ESP32-C3 and the ESP32-CAM.
- Firmware design sketch.
- Theoretical verification plan.

The following topics are **out of scope** until hardware is fabricated: measured waveforms, oscilloscope captures, thermal performance, real power consumption, and PCB bring-up results.

2 Design assumptions

The design described in this note relies on the assumptions listed in Table 1.

Table 1: Design assumptions.

ID	Area	Assumption
A1	Controller	The application node runs on an ESP32-C3-WROOM-02-N4.
A2	Vision	The ESP32-CAM is connected through UART1 (GPIO8/9) at 115200 baud.
A3	Sensors	The door contact and PIR outputs are 3.3 V logic signals; polarity is configurable in firmware.
A4	Power	The board is powered from a 5 V USB source; a 3.3 V LDO supplies the logic rails.
A5	Outputs	The buzzer is driven by a low-side N-channel MOSFET (DN2302S); the LED has a series current-limiting resistor.
A6	Host	A USB-UART channel is available for programming, debug logs, and configuration.
A7	Algorithm	The ESP32-CAM returns one of the result codes defined in the CAM protocol; the C3 does not run vision inference.
A8	Backup	No battery backup is provided; the system operates from USB power only.

3 Design artifacts

The following artifacts are part of the SecurityNode design baseline and are referenced throughout this note:

- `docs/system/system_overview.md` — System description and block diagram.
- `docs/firmware/firmware_architecture.md` — Firmware structure and build options.
- `docs/firmware/cam_protocol.md` — Binary UART protocol specification.
- `docs/hardware/` — Hardware interface notes and design rationale.
- `hardware/altium/` — Schematic and PCB source files.
- `assets/Schematic Blocks/` — Schematic-block previews.
- `assets/pcb.pdf` — PCB fabrication preview.
- This document — Application Note for the controller node.

4 Requirements

Table 2: Controller/Application Node functional requirements.

ID	Requirement	Rationale	Priority	Verification
R1	Monitor door contact continuously.	Detect unauthorized entry.	High	Inspection / Analysis
R2	Monitor PIR motion continuously.	Detect movement inside the protected area.	High	Inspection / Analysis
R3	Request visual verification from the ESP32-CAM on a sensor event.	Reduce false alarms by checking face coverage.	High	Protocol review
R4	Activate the local alarm when a suspicious condition is detected.	Deter intruders and alert occupants.	High	State-machine review
R5	Operate as a basic alarm when the vision module is unavailable.	Maintain security even if the CAM fails.	High	State-machine review
R6	Provide a debug/configuration channel over USB UART.	Allow setup, diagnostics, and logging.	Medium	Interface test
R7	Recover from transient UART errors through retry and timeout.	Improve communication robustness.	Medium	Protocol review
R8	Operate from a 5 V USB supply within standard current limits.	Simplify deployment and avoid overload.	Medium	Power budget analysis

5 Traceability matrix

Table 3 maps each requirement to the sections of this note that address it and the planned verification method.

Table 3: Requirements traceability matrix.

Req.	Addressed in	Verification activity
R1	§6.1, §8, §10	GPIO input review; state coverage analysis.
R2	§6.1, §8, §10	GPIO input review; state coverage analysis.
R3	§9, §8	Protocol inspection; state-transition review.
R4	§8, §7	Alarm-path review; output-drive analysis.
R5	§8 (cam_available flag)	State-machine review; CAM-failure scenario.
R6	§6.1, §9	USB-UART interface review.
R7	§9 (timeout/retry rules)	Protocol robustness analysis.
R8	§7	Analytical power budget.

6 External interfaces

Figure 1 shows the controller node and its surrounding interfaces.

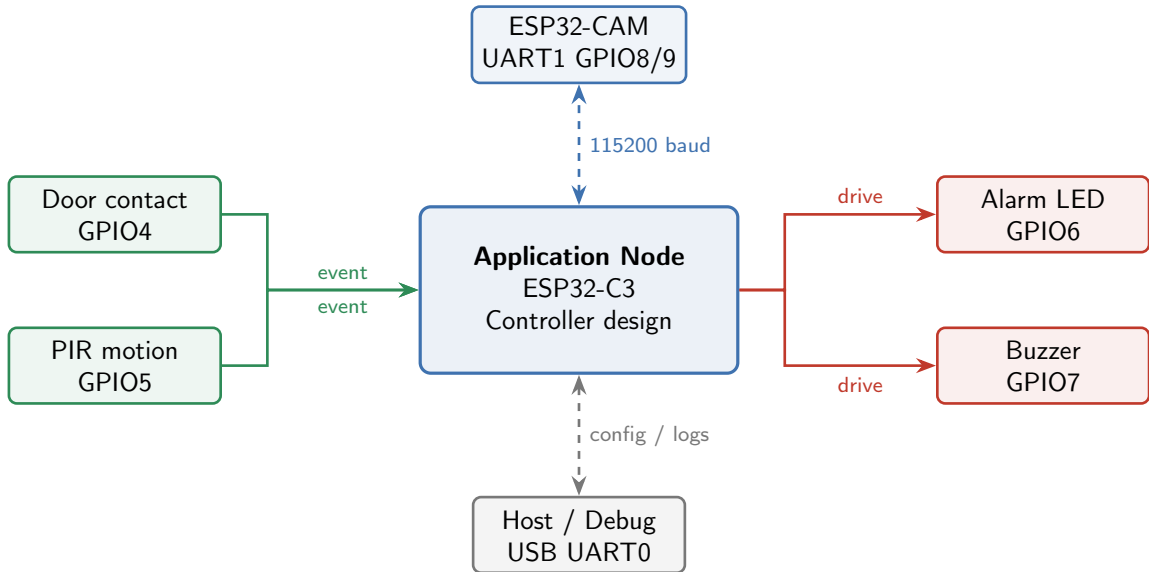


Figure 1: Application Node block diagram and external interfaces.

6.1 Pinout

Table 4 lists the controller pins used by the application node.

Table 4: Application Node pin assignment.

Signal	GPIO	Dir.	Function	Electrical notes
DOOR_IN	GPIO4	Input	Door contact state	3.3 V logic; internal or external pull; debounced in firmware.
PIR_IN	GPIO5	Input	PIR motion detect	3.3 V active-high pulse; edge or polled.
ALARM_LED	GPIO6	Output	Alarm LED drive	Active-high or active-low per schematic; series resistor limits current.
BUZZER_DRV	GPIO7	Output	Buzzer MOSFET gate	Drives DN2302S low-side switch; 5 V buzzer load.
CAM_TX	GPIO8	Output	UART1 TX to ESP32-CAM	3.3 V, 115200 baud, no flow control.
CAM_RX	GPIO9	Input	UART1 RX from ESP32-CAM	3.3 V, 115200 baud, no flow control.
USB_TX	GPIO1	Output	UART0 TX (de-bug/flash)	Routed through U5 analog switch.
USB_RX	GPIO2	Input	UART0 RX (de-bug/flash)	Routed through U5 analog switch.
3V3	—	Power	Logic supply rail	AP2112K-3.3 LDO output.
VBUS	—	Power	5 V USB input	Current-limited by TPS2553.
GND	—	Power	Common ground	Single-point/star grounding per PCB review.

6.2 UART interface parameters

Table 5: UART parameters for the ESP32-CAM link.

Parameter	Value
Controller port	UART1 (GPIO8/9)
Baud rate	115200
Data bits	8
Parity	None
Stop bits	1
Flow control	None
Frame format	Binary header/length/ID/payload/checksum

7 Electrical design summary

7.1 Power architecture

The board receives 5 V from a Mini-USB connector. A TPS2553 current-limited switch distributes power, and an AP2112K-3.3 LDO generates the 3.3 V logic rail for the ESP32-C3, ESP32-CAM, sensors, and indicator outputs. ESD protection is provided by USBLC6-2P6 and SMF5.0CA devices.

7.2 Analytical power budget

Because no hardware has been built, the budget in Table 6 is an analytical estimate used to confirm that the USB 5 V/500 mA source is adequate.

Table 6: Estimated application-node power budget.

Load	Voltage (V)	Current (mA)	Power (mW)
ESP32-C3 active	3.3	80	264
ESP32-CAM peak (Wi-Fi + camera on)	3.3	200	660
Alarm LED	3.3	10	33
Buzzer (5 V through MOSFET)	5.0	40	200
Door contact / PIR leakage	3.3	2	7
Total	—	332	1164

The estimated 332 mA is comfortably below the 500 mA nominal USB limit and the typical TPS2553 current-limit setting, leaving margin for regulator losses and inrush current.

7.3 Output drive

The buzzer is switched by a DN2302S N-channel MOSFET. The ESP32-C3 GPIO drives the MOSFET gate directly; the gate threshold and on-resistance must be checked against the buzzer current in the final schematic review. The alarm LED is driven through a series resistor sized for the desired LED current and forward voltage.

7.4 Schematic preview

The schematic is organized into the functional blocks shown in `assets/Schematic Blocks/`: power, ESP32-C3, ESP32-CAM, inputs, outputs, and UART switch. A system-level preview of these blocks is provided in the project system overview document.

7.5 PCB design review notes

Before fabrication, the PCB layout should be reviewed against the following checklist:

- Keep the UART1 traces between the ESP32-C3 and ESP32-CAM short and matched.
- Place decoupling capacitors close to each IC power pin.
- Provide a solid ground plane under the microcontroller and camera module.
- Route the buzzer load away from sensitive sensor traces to reduce switching noise.
- Confirm that the USB data and power lines have the required ESD protection and series resistors.
- Verify the U5 analog-switch routing for programming path selection.

8 State machine

The application behavior is modelled as a finite-state machine. A new state, **INCONCLUSIVE**, replaces the previous “CLEAR” state to avoid the incorrect safety implication that a visible face guarantees no threat. The **ARMED** state now maintains a `cam_available` flag that is updated by periodic status queries; this flag determines whether the system can enter the vision-verification path or must fall back to basic-alarm behavior.

8.1 State definitions

- **INIT** — System startup, self-test, GPIO/UART initialization, and initial CAM presence check.
- **ARMED** — Normal monitoring mode. LED slow-blinks, sensors are active, and the node periodically queries the CAM status to set `cam_available`.
- **TRIGGERED** — A sensor event has been detected. The LED fast-blinks. If `cam_available` is true, the node proceeds to visual verification; otherwise it enters the ALARM state as a basic alarm.
- **VISION_REQUEST** — The node sends `CMD_CAPTURE` to the ESP32-CAM and waits for an ACK.
- **VISION_CHECK** — The node waits for the analysis result while maintaining the fast-blink indication. A 5 s timeout is enforced.
- **INCONCLUSIVE** — The CAM reports a visible face, no face, or another non-threat condition. The event is logged and the node returns to ARMED without sounding the alarm.
- **ALARM** — A suspicious condition, communication failure, or CAM-unavailable condition has occurred. The LED is solid and the buzzer is ON until the node is disarmed or the alarm timeout expires.

8.2 State-transition diagram

Figure 2 shows the updated state machine.

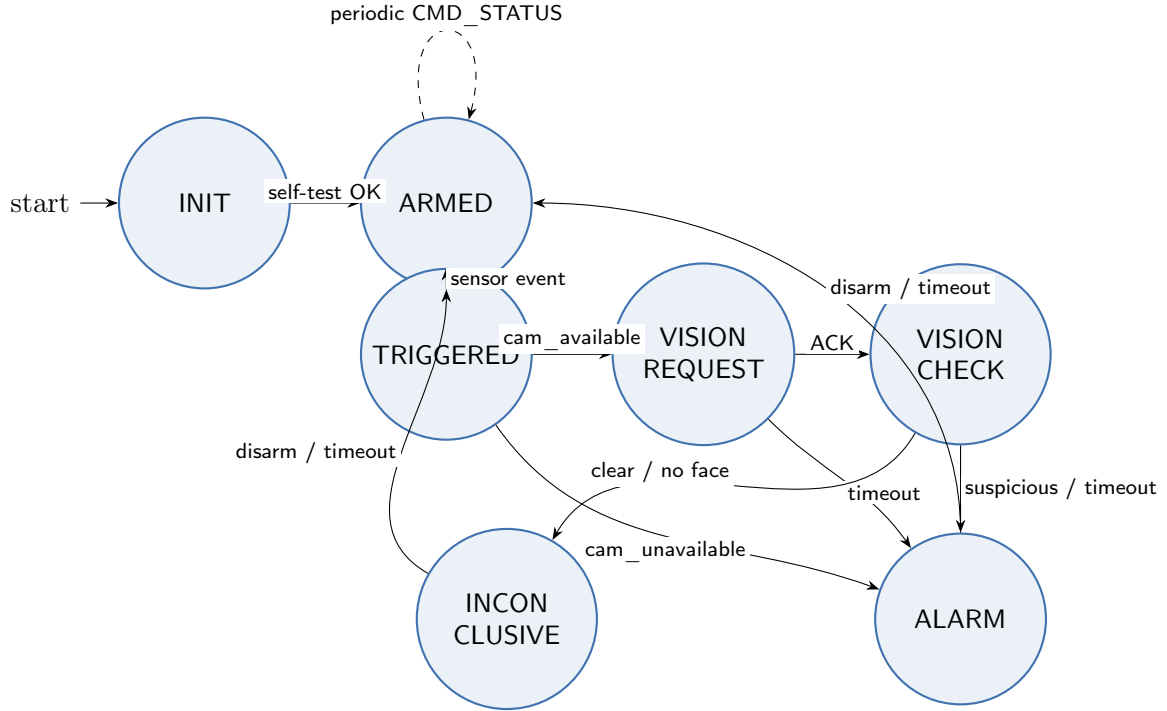


Figure 2: Application Node finite-state machine with camera-availability flag and INCONCLUSIVE state.

8.3 State behavior table

Table 7: Application Node state behavior.

State	Behavior
INIT	Power-on self-test, initialize GPIO/UART, test LED/buzzer, check CAM presence.
ARMED	LED slow blink; sensors active; periodic CAM status query updates <code>cam_available</code> .
TRIGGERED	Sensor event detected; LED fast blink; branch based on <code>cam_available</code> .
VISION_REQUEST	Send <code>CMD_CAPTURE</code> ; wait for <code>ACK</code> with 500 ms timeout and retry logic.
VISION_CHECK	Maintain fast blink; wait for <code>RSP_RESULT</code> ; enforce 5 s timeout.
INCONCLUSIVE	Non-threat result (visible face / no face); log event; return to <code>ARMED</code> .
ALARM	LED solid, buzzer ON; log event; wait for disarm or timeout.

9 UART protocol interface

The application node is the UART master for the ESP32-CAM module. Commands and responses use the binary frame defined in `docs/firmware/cam_protocol.md`.

9.1 Frame format

HEADER	LENGTH	CMD_ID	PAYLOAD	CHECKSUM
0xAA	N	1 B	N bytes	XOR

The checksum is the XOR of all bytes from LENGTH through the end of the payload.

9.2 Command set

Table 8: Application Node command set to the ESP32-CAM.

ID	Name	Purpose
0x01	CMD_CAPTURE	Request image capture and face-coverage analysis.
0x02	CMD_STATUS	Request current module status (updates <code>cam_available</code>).
0x03	CMD_RESET	Request a soft reset of the vision module.

9.3 Response handling

Table 9: Application Node response handling.

ID	Name	Action
0x81	RSP_ACK	Validate echoed command; stop retry timer.
0x82	RSP_RESULT	Evaluate result code and drive state transitions.
0x83	RSP_STATUS	Update the <code>cam_available</code> health flag.
0x84	RSP_ERROR	Log error; retry command or fall back to basic alarm.

9.4 Timeout and retry rules

Table 10: UART timeout and retry policy.

Operation	Timeout	Retries	Action on failure
ACK after command	500 ms	3	Mark CAM unresponsive; <code>cam_available=false</code> ; basic alarm.
RESULT after capture	5 s	—	Treat as suspicious; activate ALARM.
STATUS response	500 ms	2	<code>cam_available=false</code> until next successful query.

10 Firmware design sketch

The listing below is a design skeleton for the main loop. It emphasizes state updates and camera-availability handling rather than production code details.

```
void loop() {
    sensors_read(&door_state, &pir_state);

    /* Update camera health flag periodically while armed. */
    if (fsm_state == ARMED && status_query_due()) {
        cam_send_command(CMD_STATUS, NULL, 0);
    }

    fsm_update(door_state, pir_state, cam_result, cam_available);

    if (fsm_state == VISION_REQUEST && !cam_request_pending) {
        cam_send_command(CMD_CAPTURE, NULL, 0);
        cam_request_pending = true;
        timeout_start(CAM_ACK_TIMEOUT_MS);
    }

    cam_process_incoming(); /* parse ACK / RESULT / STATUS / ERROR */
    alarm_update(fsm_state); /* drive LED and buzzer */
    debug_console_tick(); /* handle USB CLI */
}
```

11 Theoretical verification plan

Because no physical prototype is available, verification is limited to analytical, inspection, and simulation-based activities.

11.1 Expected behavior and design targets

Table 11: Design targets for key behaviors.

Metric	Target	Rationale
Sensor-to-alarm latency	< 500 ms	Fast enough to deter an intruder.
ACK timeout	500 ms	Short enough to detect a stalled CAM.
Capture timeout	5 s	Allows image capture and inference without excessive delay.
Retry attempts	3	Balance robustness against UART noise with responsiveness.
Total 5 V current	< 450 mA	Maintain margin under USB 500 mA limit.

11.2 Analytical verification

- Confirm that every requirement in Table 2 is addressed by at least one design section or artifact.
- Walk through the state machine for all combinations of sensor events and `cam_available` values to ensure no undefined transitions.

- Verify the checksum algorithm for every command and response example in the protocol.
- Check the power budget in Table 6 against the regulator and current-limiter limits.

11.3 Schematic and PCB verification

- Inspect the Altium schematic against the pinout in Table 4.
- Confirm that each GPIO has the correct direction, pull, and protection.
- Review the PCB layout against the checklist in §7.
- Compare the `assets/Schematic Blocks/` previews with the final schematic before release.

11.4 Excluded verification activities

The following activities are deferred until a prototype is built:

- Oscilloscope capture of sensor and UART signals.
- Thermal imaging of the ESP32-C3, ESP32-CAM, and power components.
- Measured power consumption under idle, alarm, and capture conditions.
- PCB bring-up logs and hardware-in-the-loop tests.

12 Summary

This application note presented the design of the SecurityNode controller/application node. Key design decisions include:

- Replacing the unsafe “CLEAR” state with an **INCONCLUSIVE** state for non-threat vision results.
- Introducing a `cam_available` flag in the ARMED state so the node can gracefully fall back to a basic alarm when the ESP32-CAM is unavailable.
- Splitting the event-handling path into TRIGGERED, VISION_REQUEST, and VISION_CHECK to separate event detection, command issuance, and result waiting.
- Defining requirements, a traceability matrix, and a theoretical verification plan suitable for a design-stage review.

The node remains designed to work as a full vision-assisted alarm when the CAM is present and as a conventional sensor-driven alarm when it is not. Once hardware is available, the theoretical plan can be extended with bench measurements and bring-up results.